Éditeur collaboratif Typst — document de conception

Oct 24, 2026 · @Mathieu

Objectif et décisions

Un éditeur de bureau en Go où plusieurs personnes éditent en direct des documents Typst, Markdown et texte, sans serveur, avec trace de qui a écrit quoi, historique de versions et export PDF. Les pairs se trouvent sur le LAN ou par un code court, et l'édition continue hors ligne.

Trois décisions sont prises au 24 octobre 2026 :

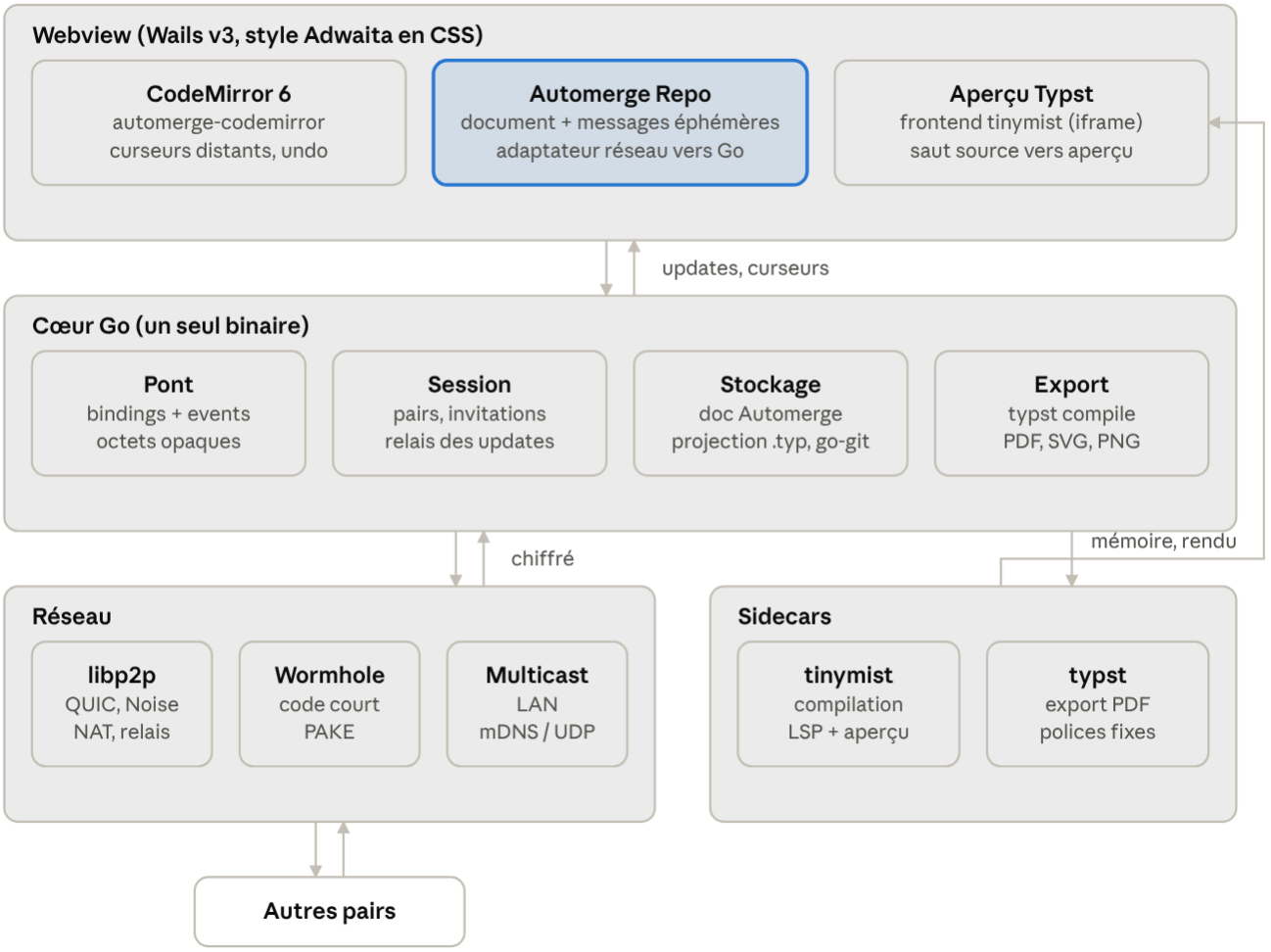
Sujet	Décision	Conséquence principale
CRDT	Automerge (`@automerge/automerge` + `automerge-repo` dans la webview, `automerge-go` côté Go si besoin)	Historique complet par auteur natif ; format binaire partagé avec Teamtype ; `automerge-codemirror` pour l'éditeur
Réseau	Pile maison en Go : `go-libp2p` (QUIC, Noise, mDNS, hole punching, relais), `wormhole-william` pour les codes, découverte LAN par multicast	Tout dans un seul binaire, licence libre, format sur disque à nous ; colle réseau à écrire et tester
Plateformes	Linux, macOS et Windows dès le départ	Wails v3, signature et notarisation, mises à jour signées, cross-compilation sans cgo à préserver autant que possible

Choix déjà arrêtés plus tôt : Wails v3 comme shell avec habillage Adwaita en CSS ; CodeMirror 6 pour la source, Tiptap en vue riche optionnelle ; tinymist en sidecar pour l'aperçu Typst et le LSP ; binaire `typst` embarqué pour l'export PDF ; go-git pour les jalons lisibles ; RTF et DOCX en import/export seulement.

Architecture

Le CRDT vit dans la webview ; Go ne connaît que des octets, le réseau et les processus annexes. Tout changement de CRDT ou de framework ne touche qu'une couche.

Automerge dans la webview, Go transporte des octets, sidecars pour Typst



architecture · 4 couches, 2 sidecars

Le document Automerge et ses messages éphémères descendent vers Go par le pont, qui les relaie aux pairs via libp2p ; tinymist reçoit le texte en fichiers mémoire et renvoie le rendu à l'iframe d'aperçu.

Couche	Paquets retenus	Rôle
Webview	`@automerge/automerge`, `@automerge/automerge-repo`, `automerge-codemirror`, CodeMirror 6, Tiptap (optionnel)	Édition, fusion, curseurs, undo par auteur
Pont	Wails v3 (bindings, events ou WebSocket local pour les gros blobs)	Transport d'octets opaques dans les deux sens
Session	`go-libp2p` (QUIC, Noise, mDNS, hole punching, relais circuit), `wormhole-william`, `schollz/peerdiscovery`	Identité Ed25519, invitations, connexion, relais des updates

Couche	Paquets retenus	Rôle
Stockage	`bbolt` ou `modernc.org/sqlite`, `google/renameio`, `fsnotify`, `go-git`	État Automerge (source de vérité), projection texte, jalons
Typst	tinymist (sidecar), binaire `typst` embarqué via `go:embed`, polices embarquées	Aperçu incrémental, LSP, export déterministe
Markdown	`@codemirror/lang-markdown`, `markdown-it`, `goldmark`, pandoc (optionnel)	Aperçu et exports
Distribution	signature + notarisation, `minio/selfupdate`, `log/slog` + `lumberjack`	Trois plateformes, mises à jour signées, logs exportables

Modèle de confiance

Décision : un modèle simple et explicite pour la v1 (invitation par code, liste de membres signée, exclusion par re-partage), en gardant la porte ouverte à Keyhive quand il sera publié. Sans serveur, le contrôle d'accès doit voyager avec les données ; c'est le point le moins mûr de tout l'écosystème local-first.

Ce que font les autres

Projet	Modèle	Révocation	Leçon
Teamtype	Un code de jonction donne l'accès complet au dossier partagé ; les pairs connus sont mémorisés	Arrêter le partage ; pas de retrait fin d'un pair	Suffit pour de petits groupes de confiance ; c'est aussi notre cas d'usage principal
Radicle	Identité = clé publique ; un dépôt a des délégués, toute référence est signée ; seuil de délégués pour les changements d'identité	Retirer un délégué par un changement d'identité signé	Signer chaque contribution rend l'auteur non répudiable : à reprendre pour les updates Automerge

Projet	Modèle	Révocation	Leçon
Keyhive (Ink & Switch)	Capacités convergentes : chaque document Automerge est identifié par une clé publique et délègue des droits à d'autres clés ; groupes transitifs ; chiffrement de bout en bout par chunks	Révocation sans coordination, gérée par un CRDT de groupe ; cas limites documentés (deux admins qui se révoquent, opérations antidatées)	C'est la réponse complète au problème ; conçu pour Automerge, donc compatible avec notre choix

Plan v1

1. Identité = paire Ed25519 par machine, affichée sous forme de nom lisible + empreinte courte. Un utilisateur peut avoir plusieurs machines ; on les regroupe sous un nom, comme le fait Keyhive avec les groupes d'appareils.
2. Invitation = code court Wormhole (PAKE) ou acceptation explicite d'un pair découvert sur le LAN. Un inconnu sur le LAN ne voit que le nom de la machine et le titre du document, jamais le contenu.
3. Chaque document porte une liste de membres signée par l'hôte initial, avec un rôle `écriture` ou `lecture`. Les pairs refusent les updates d'un auteur absent de la liste ou en lecture seule.
4. Chaque lot d'updates envoyé sur le réseau est signé par la clé de l'auteur. Automerge identifie déjà l'auteur par `actorId` ; la signature empêche de se faire passer pour quelqu'un.
5. Révocation v1 = retirer le membre de la liste et faire tourner la clé de session (nouvelle clé symétrique partagée aux membres restants). Ce qu'il a déjà reçu reste chez lui : c'est inévitable, Keyhive le dit aussi.
6. Pas de chiffrement au repos ni de relais non-fiables en v1 ; la connexion est déjà chiffrée par Noise. Dès que Keyhive atteint une version stable avec liaison JS, l'évaluer pour remplacer les points 3 à 5.

Questions ouvertes

- Que fait un pair avec des updates reçues hors ligne d'un auteur révoqué entre-temps ? Proposition : les accepter si elles précèdent causalement la révocation, les signaler sinon.
- Faut-il un rôle `admin` distinct de l'hôte initial pour qu'un document survive au départ de son créateur ?

Ressources binaires et paquets Typst

Décision : les binaires ne vont jamais dans le document Automerge ; ils sont adressés par hachage et transférés à la demande. Les paquets Typst sont vendus dans le projet à l'import, pour que la compilation ne dépende jamais du réseau.

Images, polices, bibliographies

- Le document Automerge ne contient que des références : `{ path: "figures/plan.png", hash: "blake3:…", size: 184320 }`. Automerge compresse mal les gros blobs et toute version d'une image resterait dans l'historique pour toujours.
- Un magasin de blobs par projet, adressé par contenu (BLAKE3, comme iroh-blobs). Deux pairs comparent les hachages manquants à la connexion et se les transmettent sur un flux libp2p dédié ; une image remplacée n'est qu'un nouveau hachage.
- Teamtype évite le sujet en ne synchronisant que les fichiers texte ; typst.app stocke les ressources côté serveur. Notre cas est entre les deux : il faut le magasin de blobs, mais un quota par fichier (par exemple 25 Mo) et par projet évite de transformer l'éditeur en outil de transfert.
- Les blobs sont écrits dans le dossier projet sous leur chemin lisible pour que tinymist, le CLI `typst` et git les voient ; `fsnotify` déclenche la recompilation. Les polices du projet passent par le même magasin et par `--font-path`.

Paquets Typst hors ligne

Le problème est réel : Typst télécharge les paquets à la compilation sans moyen officiel de les pré-télécharger, ce qui bloque la compilation hors ligne et les environnements en lecture seule. Trois outils ont le même remède :

Outil	Approche
typvend	Scanne les imports `@preview/<pkg>:<version>` des `.typ` et télécharge les paquets dans un dossier, pointé via `--package-cache-path`
Quarto	Regroupe les paquets dans le projet (`typst/packages/`), les met en scène avant `typst compile` et les passe via `--package-path` pour `@local` et `--package-cache-path` pour `@preview` ; le dossier est versionné avec le projet
typst-pack	Produit un fichier unique contenant sources, ressources, paquets et polices, avec un mode `offline` qui échoue si un paquet manque

Plan : à l'ouverture ou à chaque import ajouté, Go scanne les `@preview/…` du projet, télécharge les versions exactes dans `<projet>/.typstdeps/` (structure du cache Typst), et lance tinymist et `typst` avec `--package-cache-path` sur ce dossier. Ce dossier est synchronisé entre pairs comme des blobs ordinaires, donc un pair qui rejoint hors ligne a déjà tout. Un paquet introuvable produit un message explicite dans l'éditeur plutôt qu'une erreur de compilation brute.

Idée reprise de typst-pack : offrir « Exporter le projet » en archive autonome (sources + blobs + paquets + polices), utile pour archiver ou envoyer à quelqu'un sans l'application.

Fusion, historique et révision

Décision : reprendre le modèle « formalité à la demande » de Patchwork (Ink & Switch), construit précisément sur Automerge : on édite directement, l'historique est auto-sauvé à chaque frappe, et la structure (jalons, groupes d'éditions, branches) s'ajoute après coup, jamais avant. Le choix d'Automerge rend tout ceci presque gratuit.

Pourquoi Automerge rend ça simple : il ne supprime jamais rien et stocke chaque changement compressé ; il calcule un diff exact entre deux points de l'historique sans heuristique, chaque caractère ayant son identifiant ; et il sait cloner puis refusionner un document, ce qui porte le branchement. Patchwork a aussi obtenu l'attribution des changements aux auteurs dans les diffs, fonctionnalité reversée en amont.

Ce que Patchwork a testé, et ce qu'on reprend

Expérience	Résultat rapporté	Pour nous
Historique dynamique : auto-save de chaque frappe, vues regroupées par auteur et par période, jalons posés à la main	Un petit jeu de regroupements (auteur, temps) couvre presque tous les besoins	Panneau « Activité » v1 : lignes groupées par auteur et par session, bouton « marquer un jalon » qui crée aussi le commit go-git
Diff dans l'éditeur : ajouts surlignés, suppressions remplacées par un glyphe à survoler, remplacements « avant → après » dans la marge	Préféré au barré classique, moins bruyant	Mode « voir les changements depuis ma dernière synchro » ; c'est la réponse à la fusion laide après hors-ligne

Expérience	Résultat rapporté	Pour nous
Diff résumé : « minibar » horizontale montrant où dans le document ça a changé, stats par section, comptes en mots plutôt qu'en caractères	La minibar gagne sur les longs documents	Minibar sous l'éditeur, colorée par auteur
Groupes d'éditions : éditer directement, puis fournir rétroactivement une raison et regrouper des éditions liées ; l'auteur principal annule ce qu'il refuse	Revenir sur des éditions récentes hors ordre chronologique s'est révélé très utile ; le regroupement atomique peut-être moins	Annulation sélective par auteur dans le panneau Activité ; les groupes en v2
Branches simples : créées depuis `main` seulement, sans nom obligatoire, créables rétroactivement à partir de la session en cours, supprimées à la fusion	Convaincant pour l'exploration individuelle et pour envoyer une proposition à un coauteur	Branche = clone Automerge ; v2, après les commentaires
Historique comme fil de discussion : résumés automatiques des éditions, notes libres, fusions et jalons comme événements marquants	Les résumés courts générés sont la vue la plus utile de toutes ; le chat sert de message de commit sans cérémonie	Notes libres dans l'Activité en v1 ; résumés automatiques optionnels et locaux en v2

Leçons d'Upwelling (le projet précédent, 2023) : la collaboration en direct peut stresser les auteurs qui se sentent observés ; Upwelling proposait des brouillons privés fusionnés quand leur auteur est prêt. Patchwork a jugé la couche de brouillon obligatoire trop formelle. Compromis retenu : un bouton « écrire en privé » qui ouvre une branche locale non diffusée, désactivé par défaut.

Undo : par auteur uniquement, via l'historique Automerge filtré sur son `actorId` ; l'undo global n'existe pas.

Idée à nous : relier la minibar au PDF. Cliquer sur une zone modifiée fait sauter l'aperçu Typst à la page correspondante grâce aux spans de tinymist, pour relire le rendu de ce qui a changé plutôt que la source.

La révision telle que l'utilisateur la voit

La section précédente dit *comment* ; celle-ci dit *ce que la personne voit*, en reprenant les gestes connus de Word sans ses défauts. Règle d'or : aucun mot technique dans l'interface. On ne parle ni de CRDT, ni de commit, ni de branche en v1 ; on parle de **modifications**, de **versions** et de **personnes**.

Suivi des modifications, à la Word mais toujours allumé

Dans Word, il faut penser à activer le suivi, et un oubli perd tout. Ici, chaque frappe est enregistrée avec son auteur et l'heure, donc le suivi est toujours actif ; ce qui s'allume ou s'éteint, c'est seulement l'*affichage*.

- Bouton « Afficher les modifications » dans la barre d'en-tête, avec un menu : *depuis ma dernière visite* (par défaut), *depuis la dernière version nommée*, *depuis une date*, *par une personne*.
- Rendu : ajouts dans la couleur de l'auteur, suppressions repliées en une petite marque dans la marge qu'on survole pour lire le texte supprimé (le barré de Word rend le document illisible), remplacements montrés « avant → après » dans la marge. On peut à tout moment lire le texte propre.
- Sur chaque modification, au survol : « Marie, hier 16:42 » et, si elle en a donné une, la raison. Deux actions : **Garder** et **Annuler**. Annuler ne touche qu'à cette modification, pas à ce que d'autres ont écrit autour.
- Boutons globaux « Tout garder » et « Parcourir » (suivant / précédent, comme dans Word).
- Quand on a fini de relire, « Marquer comme lu » : les couleurs disparaissent jusqu'aux prochains changements.

Le « pourquoi »

Personne ne remplit un message de commit. Trois façons douces d'obtenir une raison :

1. Quand on termine une séance de travail (fermeture, ou pause de 20 minutes), une petite ligne propose « Résumer ce que vous venez de faire (facultatif) » ; une phrase suffit, elle s'attache à toutes les modifications de la séance.
2. Dans la vue des modifications, on peut sélectionner plusieurs changements et leur donner une raison après coup (« Reformulation de l'introduction »).
3. À défaut, l'application fabrique un résumé factuel et neutre : « Marie a modifié 3 paragraphes de la partie 2 et ajouté une figure » ; c'est généré localement à partir du diff et du plan, jamais envoyé ailleurs.

Versions nommées

- « Enregistrer une version » (Ctrl/Cmd+Maj+S) : un nom libre (« Envoyé au client », « v1.2 »), optionnellement une note ; elle apparaît comme une étape dans l'historique. Rien n'est figé : on continue d'écrire.
- Comparer deux versions nommées : même rendu coloré que ci-dessus, par auteur.
- Exporter une version : PDF daté avec le nom de la version dans les métadonnées.
- Sous le capot, c'est un jalon go-git, mais le mot « commit » n'apparaît jamais.

Revenir en arrière

- Panneau « Historique » : une frise verticale, du plus récent au plus ancien, où chaque entrée est une phrase : « Aujourd'hui 10:12 — Paul a ajouté le chapitre Sécurité (12 paragraphes) », « Hier — Version : Relecture terminée ». Les séances sont groupées par personne et par moment, pas une ligne par frappe.
- Un curseur temporel au-dessus de l'éditeur : on le fait glisser, le document se réécrit sous les yeux à cette date, en lecture seule, avec la date affichée en gros. Les autres continuent de travailler, on ne les dérange pas.
- Depuis un point du passé, trois actions, chacune expliquée en une phrase : **Restaurer ce paragraphe** (ramène juste ce morceau), **Restaurer tout le document à ce moment** (crée une nouvelle modification, l'historique n'est pas effacé, les autres la voient arriver comme n'importe quel changement), **Copier le texte**.
- Rien ne se perd jamais : « annuler une restauration » est une restauration de plus. On le dit explicitement dans l'interface pour enlever la peur de casser quelque chose.

Qui a écrit quoi, sans rien demander

- Mode « Auteurs » (icône de personnes) : le document se colore par auteur, comme la fonction d'attribution qu'Ink & Switch a tirée de l'historique Automerge ; une légende indique la part de chacun. Utile pour savoir à qui poser une question sur un passage.
- Clic droit sur un paragraphe → « Qui a écrit ceci ? » : la liste des personnes et dates, avec un lien vers la séance dans l'historique.

Ce qu'on ne fait pas, volontairement : pas de mode « suggestion » séparé (relire, c'est écrire, puis garder ou annuler), pas d'acceptation obligatoire pour que le document soit « propre », pas de vocabulaire de versionnage. Si un jour les branches arrivent, elles s'appelleront « brouillons » et se présenteront comme une version nommée qu'on n'a pas encore fusionnée.

Divergences : quand deux personnes ont voulu deux choses

Techniquement il n'y a jamais de conflit : Automerge fusionne toujours et le document reste un seul texte. Mais il y a des **divergences** : deux personnes ont réécrit la même phrase hors ligne, l'une a supprimé un paragraphe que l'autre complétait, deux ont

renommé le même fichier, deux ont remplacé la même image. Le logiciel ne doit ni les cacher (le lecteur découvre une phrase mélangée) ni bloquer (personne n'attend qu'un conflit soit « résolu » pour continuer). Il les **signale, les attribue et les règle à deux clics**.

Détection

Une divergence est déclarée quand deux séries de modifications concurrentes (ni l'une ni l'autre ne connaissait l'autre) touchent la même zone : même phrase ou paragraphe pour le texte, même entrée pour l'index du projet, même fichier pour un blob. Le texte fusionné reste tel quel dans le document, mais la zone est marquée.

Ce que l'utilisateur voit

- Dans l'éditeur : un liseré orange dans la marge et un fond très léger sur la zone, avec une étiquette « Marie et Paul ont modifié ce passage en même temps ». Le texte reste lisible et éditable ; corriger la phrase à la main règle la divergence tout seul.
- Carte de divergence au clic : les deux versions côte à côte, chacune avec l'auteur, l'heure et la raison si elle existe, et quatre boutons en langage simple : **Garder celle de Marie**, **Garder celle de Paul**, **Garder les deux** (l'une après l'autre), **Je reprends à la main** (ouvre la zone en édition avec les deux textes visibles). Un champ de commentaire est attaché : la discussion a lieu sur place.
- Dans l'indicateur d'état : « 2 divergences à régler » cliquable ; dans l'Activité : une entrée par divergence, ouverte ou réglée, avec qui l'a réglée et comment ; dans l'explorateur : une pastille orange sur le fichier concerné.
- Pour les binaires : les deux fichiers sont conservés (« plan.png » et « plan (version de Paul).png »), la carte montre les deux images, on choisit ; aucun fichier n'est écrasé.
- Pour les renommages ou déplacements : les deux noms sont proposés, le fichier n'a jamais disparu.
- Pour les membres (deux admins ont changé un rôle en même temps) : la règle déterministe s'applique immédiatement pour que tout le monde voie la même liste, et une carte en informe les admins, qui peuvent corriger.

Qui a le droit de trancher

Un réglage de projet, en trois options claires, choisi à la création et modifiable par un admin :

1. *Tout le monde* (défaut pour les petits groupes) : toute personne en écriture peut régler une divergence. Le réglage est journalisé.
2. *Les auteurs concernés et les admins* : seuls Marie, Paul et les admins voient les boutons ; les autres voient la carte, peuvent commenter et peuvent proposer un règlement, qui apparaît comme une suggestion à valider.

3. *Une personne désignée* : un « responsable du document » (ou plusieurs) règle les divergences ; les autres proposent. Correspond au relecteur unique ou au chef de projet.

Dans tous les cas :

- Régler une divergence est une modification ordinaire : elle a un auteur, une heure, une raison, elle s'annule comme le reste et la version écartée reste dans l'historique. On l'écrit dans la carte : « Rien n'est perdu, l'autre version reste consultable ».
- Si deux personnes règlent la même divergence en même temps, c'est… une divergence, traitée pareil. En pratique la carte montre qui est en train de la regarder (présence), ce qui suffit à l'éviter.
- Une divergence non réglée n'empêche rien : ni l'export, ni la version nommée ; l'export avertit seulement « 2 divergences non réglées, exporter quand même ? ». Le passage est exporté tel qu'il est dans le document.
- Un délai configurable (par défaut 7 jours) fait passer une divergence non réglée en « ancienne » : plus de liseré, elle reste dans l'Activité. Le document ne reste pas orange à vie.

Éviter la divergence avant qu'elle arrive

- La présence fait l'essentiel : voir le curseur de l'autre dans le paragraphe suffit en direct. Les divergences viennent du hors-ligne.
- Au retour en ligne, avant de se mettre à écrire, une bannière : « Pendant votre absence, Marie a modifié 3 passages, dont 1 que vous avez aussi changé » avec « Voir », pour relire avant de continuer.
- « Réserver ce passage » (clic droit) : marqueur consultatif, visible par tous (« Paul travaille sur cette section jusqu'à ce soir »), qui n'empêche personne d'écrire mais qui prévient. Jamais de vrai verrou : un verrou qu'on oublie bloque un projet, un avertissement qu'on ignore est juste un avertissement.

Ce que font les autres : Word gère le conflit à la sauvegarde avec un dialogue bloquant ; Google Docs n'en a pas (il n'y a pas de hors-ligne long) ; git impose la résolution avant de continuer ; Patchwork montre le diff et laisse l'auteur principal annuler ce qu'il refuse. Notre position est celle de Patchwork avec, en plus, l'attribution d'un droit de trancher pour les équipes qui en ont besoin.

Commentaires, relecture, lecture seule

Décision : les commentaires entrent dans la v1, stockés dans le document Automerge lui-même et ancrés par des curseurs stables ; le mode lecture seule est un rôle du modèle de confiance, appliqué par les pairs, pas par l'interface.

Ancrage : un curseur Automerge est une position relative, « avant le caractère X », qui reste stable quand le document est édité ; sa représentation texte est partageable. Un commentaire = `{ id, auteur, texte, anchorStart, anchorEnd, résolu, réponses[] }` dans une liste Automerge à côté du texte, avec un paramètre `move_cursor` qui dit où atterrit l'ancre si son caractère est supprimé. Les marques (`marks`) servent à surligner la plage commentée dans CodeMirror, avec un comportement d'expansion défini si on insère au bord.

Ce que Patchwork a appris sur les commentaires

- Les interactions de commentaire (saisir, répondre, résoudre) sont identiques pour toute application ; ce qui change, c'est la chose commentée, décrite par un « pointeur » : une plage de texte, des cellules, des formes. Pour nous : un type `Pointer` qui désigne une plage de source Typst ou Markdown ; plus tard, une zone du PDF rendu (page + rectangle), reliée à la source par les spans de tinymist.
- Les mêmes pointeurs alimentent les diffs : l'éditeur reçoit une liste de pointeurs « modifié » et les surligne. Un seul mécanisme pour deux usages.
- Un raccourci unique partout (`Cmd/Ctrl+Shift+M`) et une liste linéaire des commentaires que le relecteur lit de haut en bas, même quand ils sont ancrés dans l'espace.
- Patchwork n'a pas tranché la relation entre commentaires sur le texte et commentaires sur l'historique ; on garde les deux dans le même panneau Activité, triés par date, pour voir si la distinction compte.

Relecture : la combinaison « éditer directement + voir le diff par auteur + annuler ce qu'on refuse » (groupes d'éditions) remplace le mode suggestion de Google Docs et le suivi des modifications de Word. Un relecteur peut donc écrire dans le document ; l'auteur voit ses changements en couleur et les garde ou les annule. Les branches, en v2, offrent la variante « proposer sans toucher ».

Lecture seule : le rôle `lecture` de la liste de membres. Les pairs ignorent les updates de texte d'un membre en lecture, mais acceptent ses commentaires et ses curseurs ; l'éditeur passe en non éditable chez lui pour éviter la frustration d'écrire dans le vide. Teamtype n'a pas ce rôle ; typst.app et Google Docs l'ont et c'est la première demande des relecteurs externes.

Idée à nous : un commentaire sur le PDF. Le relecteur clique dans l'aperçu rendu, tinymist donne le span source, le commentaire s'ancre dans la source. Les gens relisent le rendu, pas le code.

Projets, sidecars, accessibilité

Projets et sessions

- L'unité de partage est le projet (un dossier), comme Teamtype et typst.app, pas le fichier : un document Typst a ses images, sa bibliographie, ses paquets. Un `Repo` Automerge par projet, un document Automerge par fichier texte, un index Automerge du projet (arborescence, membres, blobs).
- Plusieurs projets ouverts = plusieurs fenêtres Wails, chacune avec son `Repo`, ses connexions et ses sidecars ; un même pair peut être connecté à plusieurs projets.
- L'édition externe (VS Code ouvert sur le même dossier) est tolérée : `fsnotify` détecte le changement, `sergi/go-diff` le convertit en `splice` Automerge. Documenté comme mode dégradé : pas d'attribution fine, pas de curseur.

Sécurité des sidecars (tinymist, `typst`, un jour pandoc)

Le modèle repris est celui des SDK à moteur embarqué : binaire signé et notarisé sur macOS, mis en cache par version, vérifié contre les checksums à chaque démarrage, et arrêté avec le processus parent. Pour nous :

1. Les binaires sont embarqués (`go:embed`) avec leur SHA-256 compilé dans l'application ; extraction dans `os.UserCacheDir()/<app>/<version>/`, vérification du hachage avant chaque lancement, fichiers en lecture seule.
2. Tout serveur local (plan de données tinymist, WebSocket local du pont) écoute sur `127.0.0.1` à un port éphémère et exige un jeton aléatoire généré au lancement, comparé en temps constant et jamais journalisé. Un autre programme sur la machine ne doit pas pouvoir lire le document par le port de l'aperçu.
3. Compilation bornée : `context.WithTimeout` (30 s pour un export), processus séparé donc limite mémoire possible par cgroup ou job object, relance automatique avec backoff si tinymist meurt, et jamais plus d'une relance par minute sans message à l'utilisateur.
4. Le binaire `typst` compile avec `--root` sur le dossier projet : un `#read("../../.ssh/id_ed25519")` glissé par un pair est refusé par Typst lui-même. À vérifier aussi pour tinymist (option équivalente).
5. Arrêt propre : les sidecars sont des enfants du processus et reçoivent SIGTERM à la fermeture de la fenêtre ; sur Windows, un job object pour qu'ils meurent avec l'application.

Accessibilité et i18n

- CodeMirror 6 est utilisable au lecteur d'écran et au clavier ; Adwaita en CSS doit garder les contrastes et les tailles de cible (44 px) du vrai libadwaita, et respecter `prefers-reduced-motion` et `prefers-color-scheme`.
- Interface en français et anglais dès la v1 ; chaînes dans des fichiers JSON par langue côté frontend, `golang.org/x/text/message` côté Go pour les messages d'erreur. Noms d'auteurs, dates et nombres formatés par `Intl`.

- Raccourcis : la palette de commandes de CodeMirror couvre l'éditeur ; une palette d'application (`Ctrl/Cmd+K`) couvre partager, inviter, exporter, jalonner, commenter. Tous les raccourcis sont listés et modifiables.
- Couleurs des pairs : palette de 8 teintes distinguables en vision daltonienne (Okabe-Ito), le nom toujours affiché au survol, jamais la couleur seule comme information.

Tests de convergence et simulation réseau

Décision : ne pas tester Automerge (c'est leur travail, et il est fuzzé), tester *notre* colle autour avec une simulation déterministe à graine, et un harnais de rejeu d'historique réel.

Ce que font les projets CRDT sérieux

- Un harnais de simulation déterministe : plusieurs agents émettent des événements sur un réseau à fautes injectées (pertes, réordonnancement, duplication, partitions, dérive d'horloge) ; après drainage, un oracle vérifie convergence, commutativité, idempotence et cohérence causale ; chaque graine est rejouable et une graine en échec donne la trace complète.
- La même idée chez Calimero : un `NetworkRouter` simulé avec injection de fautes et gestion de partitions remplace libp2p, une horloge discrète et un RNG à graine rendent les échecs reproductibles, sans I/O réseau réelle.
- Tests par propriétés sur des séquences d'opérations aléatoires sur N répliques, fusionnées dans tous les ordres (proptest, équivalent Go : `pgregory.net/rapid` ou `testing/quick`).

Plan pour nous

1. *Interface* `Transport` : tout le réseau passe par une interface Go minuscule (`Send(peer, bytes)` , `Recv() <-chan msg` , `Peers()`). La version production est libp2p ; la version test est un routeur en mémoire avec fautes injectées et horloge virtuelle. C'est le prérequis de tout le reste.
2. *Simulation à graine* : N pairs virtuels, chacun pilotant un vrai document Automerge (via `automerge-go` , sans webview), éditions aléatoires, fautes aléatoires, puis drainage et vérification : textes identiques, `Heads` identiques, aucune update refusée à tort par la couche de confiance. Lancée en CI sur 200 graines, graine imprimée à l'échec, commande `replay --seed` .
3. *Scénarios nommés* : hors-ligne long puis retour, révocation concurrente, deux admins, pair qui rejoint un document de 50 000 opérations, perte de l'hôte initial, mêmes phrases éditées des deux côtés. Chacun est un test lisible, pas seulement une graine.

4. *Rejeu d'historique réel* : un fichier `.automerge` exporté depuis une vraie session (anonymisé) est rejoué change par change dans un pair neuf ; on vérifie que la projection `.typ` sur disque est identique à chaque étape. C'est le test de non-régression pour la persistance et la projection.
5. *Tests de bout en bout* : deux instances de l'application lancées par le harnais Wails sur la même machine, reliées par libp2p en boucle locale, pilotées par Playwright dans la webview. Peu nombreux, lents, mais ce sont les seuls qui attrapent les bugs du pont.
6. *Fuzz Go natif* (`go test -fuzz`) sur les décodeurs : messages réseau, liste de membres signée, fichiers de projet. Tout ce qui lit des octets venus d'un pair est fuzzé.

Idée reprise : la trace d'une graine en échec est le meilleur rapport de bug possible ; la conserver aussi en production (journal circulaire des 1 000 derniers messages réseau, exporté avec les logs) pour rejouer un incident utilisateur en simulation.

Confidentialité, licence, dépendances

Promesse de confidentialité : à écrire en une page dans l'application et sur le site, dès la v1, sur le modèle d'Obsidian : aucune télémétrie ; la seule connexion sortante est la vérification de mise à jour, qui n'envoie que le numéro de version. L'histoire d'Obsidian montre aussi qu'une application locale sans politique publiée inquiète les utilisateurs, même quand elle ne collecte rien ; une politique courte qui dit « nous ne collectons pas » vaut la peine d'être publiée.

Ce que notre page doit dire, point par point :

Flux	Qui voit quoi	Contrôle
Multicast LAN	Les machines du même réseau voient le nom de la machine et les titres des documents ouverts au partage	Désactivable ; aucune annonce tant qu'aucun document n'est partagé
Code d'invitation	Le serveur de rendez-vous Wormhole voit deux adresses IP et un code hashé, jamais le contenu	Serveur configurable ; auto-hébergeable
Relais libp2p	Un relais voit les adresses et le volume, pas le contenu (chiffré Noise de bout en bout)	Désactivable (LAN seulement) ; relais configurable
Paquets Typst	Le registre Typst Universe voit les paquets téléchargés	Pré-téléchargement à l'import, puis plus rien

Flux	Qui voit quoi	Contrôle
Mise à jour	Notre serveur voit la version et la plateforme	Désactivable ; signatures vérifiées
Résumés d'historique (v2)	Rien ne sort : modèle local ou fonction désactivée	Jamais de service distant par défaut

Licence du projet : décision à prendre, elle conditionne les dépendances. Avec la pile maison (sans Teamtype), toutes les dépendances retenues sont MIT, Apache-2.0 ou BSD : Automerge (MIT), go-libp2p (MIT/Apache), wormhole-william (MIT), CodeMirror (MIT), tinymist et typst (Apache-2.0), Wails (MIT). Aucune ne contraint la licence de l'application. Si on vend l'application un jour, cela reste possible ; si on la publie en libre, MPL-2.0 ou AGPL selon qu'on veut ou non empêcher une version SaaS fermée.

Dépendances portées par des bénévoles et leur plan B :

Dépendance	Risque	Mitigation
tinymist	Un mainteneur principal, rythme très rapide	Épingler une version par release de l'app ; le remplaçant est `typst watch` + rendu SVG (dégradé mais fonctionnel)
Automerge	Financé par Ink & Switch, équipe réduite	Format binaire ouvert et spécifié ; `automerge-go` permet de relire les fichiers sans la webview
wormhole-william	Un auteur	Protocole Magic Wormhole stable et multi-implémentations ; le remplaçant est `schollz/pake` + notre relais
go-libp2p	Protocol Labs, large base d'utilisateurs	Faible ; alternative `tsnet` derrière la même interface `Transport`
Wails v3	Bêta, communauté active	Couche shell isolée ; MyGo ou GTK comme sorties de secours

Épinglage : `go.mod` et `package-lock.json` épinglés à la version exacte ; binaires des sidecars avec hachage dans le dépôt ; Renovate ou Dependabot en mode « proposer, ne jamais fusionner seul » ; chaque montée de version de tinymist ou typst déclenche le rejeu d'historique et la comparaison PDF du jeu de documents de référence.

Solutions clé en main intégrables

Une bonne partie des problèmes listés plus haut a déjà une brique finie, directement importable dans la pile Go + webview retenue. Le tableau donne, pour chaque problème, la brique, ce qu'il reste à écrire, et la réserve.

Problème	Brique clé en main	Ce qu'il reste à écrire	Réserve
Synchro multi-documents, stockage, partage sélectif	`@automerge/automerge-repo` : gestion de nombreux documents, protocole de synchro, adaptateurs réseau et stockage enfichables, `NetworkAdapter` comme classe de base pour un transport maison, politique de partage personnalisable	Un `NetworkAdapter` d'environ 100 lignes qui parle au pont Wails ; un `StorageAdapter` qui écrit via Go	Aucune : c'est l'usage prévu
Pont webview ↔ Go pour les octets Automerge	`@automerge/automerge-repo-network-messagechannel` comme modèle ; côté Go, `coder/websocket` sur `127.0.0.1` avec jeton	Le mapping messages ↔ WebSocket	Mesurer le débit à l'étape 1
Diff « depuis ma dernière synchro », revue et acceptation chunk par chunk	`@codemirror/merge` : vue côte à côte et vue unifiée inline, commandes pour naviguer entre changements et accepter ou rejeter chaque chunk, `updateOriginalDoc` pour changer la base comparée	Fournir la version de base depuis un `Heads` Automerge ; colorer par auteur via les marques	Officiel CodeMirror, MIT ; 1 800 lignes
Marqueurs de gouttière « modifié / ajouté / supprimé » façon VS Code	`@fazelstudio/codemirror-gitgutter` : fonction pure de (base, document), vue « peek » inline, revert local	Brancher la base sur l'historique Automerge	Jeune, un auteur ; l'alternative est la vue unifiée de `@codemirror/merge`
Autocomplétion, diagnostics, survol	`codemirror-languageserver` (client LSP pour CodeMirror 6) + tinymist	Transport LSP via le pont (stdio du sidecar relayé en WebSocket)	Paquet communautaire, API stable
Transfert des blobs (images, polices, paquets) par hachage	`github.com/ipfs/boxo/bitswap` : échange de blocs adressés par contenu sur libp2p, avec réseau de test simulé à latence et débit réglables	Un `blockstore` sur le dossier projet	Lourd (dépend de l'écosystème IPFS) ; pour un petit nombre de pairs, un protocole libp2p maison de 200 lignes peut suffire. À trancher par un essai

Problème	Brique clé en main	Ce qu'il reste à écrire	Réserve
Simulation réseau déterministe	`go-libp2p/p2p/net/mock` : pairs virtuels, liens avec latence et bande passante, déconnexion et reconnexion, maillage complet en une ligne	Les scénarios et l'oracle de convergence	Fourni par libp2p lui-même ; notre pile de production tourne telle quelle dessus
Capacités et délégation signées (modèle de confiance)	`ucan-wg/go-ucan` : délégation et invocation UCAN, DID did:key, conteneurs CBOR/CAR, licence Apache-2.0 + MIT	Définir trois capacités (`lire` , `écrire` , `inviter`) sur une ressource `doc:<id>`	Les UCAN sont des chaînes de certificats sans état : la révocation reste à nous (liste signée) ; c'est exactement la limite que Keyhive veut dépasser
Habillage Adwaita de la webview	adwaita.css : feuille qui donne aux éléments HTML natifs le style libadwaita ; palette libadwaita 1.6+ documentée (thème Obsidian « Adwaita Modern ») ; polices Adwaita Sans et Adwaita Mono libres	Composants spécifiques (barre d'en-tête, barre latérale, toasts)	Petits projets communautaires ; reprendre les variables de couleur plutôt que dépendre du fichier entier
Éditeur Markdown collaboratif riche (v2)	`automerge-prosemirror` (officiel) + Tiptap	Les boutons	Même modèle que `automerge-codemirror`
Codes d'invitation	`psanford/wormhole-william`	Écran de saisie	Un auteur, protocole stable
Découverte LAN	`schollz/peerdiscovery` ou le service mDNS intégré de libp2p (`p2p/discovery/mdns`)	Le payload d'annonce	Préférer mDNS libp2p : une dépendance de moins
Historique, commentaires, groupes d'éditions	Le code des prototypes Patchwork (Tiny Essay Editor) est publié par Ink & Switch : vues d'historique, diffs attribués par auteur, commentaires ancrés, le tout sur Automerge	Extraction des composants utiles ; ce sont des prototypes React	À vérifier : licence et état du dépôt avant de s'appuyer dessus ; au minimum une référence d'implémentation
Mises à jour signées	`minio/selfupdate` (vérification Ed25519 ou SHA-256)	Le serveur de releases	Mûr
Sidecar vérifié et supervisé	Pas de brique Go dédiée ; `go:embed` + `crypto/sha256` + `os/exec` suffisent, modèle Hatchet	~150 lignes	—

Ce qui reste sans brique : le panneau Activité (regroupement par auteur et session), la minibar de diff, les commentaires ancrés dans le PDF, la liste de membres signée avec révocation, et la vente des paquets Typst depuis Go. Ce sont de petits modules, et ce sont aussi ceux qui différencient le produit.

Effet sur le plan : l'étape 1 passe par `automerge-repo` + un `NetworkAdapter` maison plutôt que par Automerge nu, l'étape 4 commence par `@codemirror/merge`, et l'étape 5 par `mocknet`. Rien d'autre ne change.

Zones d'ombre et comment d'autres les ont résolues

Dix points où le plan repose encore sur une hypothèse. Pour chacun : ce qui est flou, ce que d'autres ont fait, ce qu'on en retient.

1. L'undo avec Automerge. Automerge n'a pas d'UndoManager intégré, contrairement à Yjs et Loro. Le wrapper communautaire `automerge-repo-undo-redo` prévient lui-même qu'il altère l'historique du document, avec des résultats possiblement inattendus. Loro formule la bonne règle : l'undo doit être local, ne défaire que ses propres opérations sans toucher celles des autres, et se réalise en supprimant et recréant les éléments plutôt qu'en remontant le temps. Retenu : undo local par opérations inverses (un nouveau `splice` qui annule le nôtre), positions suivies par curseurs Automerge, jamais de réécriture d'historique. L'historique de CodeMirror fait déjà cela si les changements distants sont marqués hors historique ; à valider à l'étape 1.

2. L'entrelacement de texte. Deux personnes qui tapent au même endroit peuvent voir leurs lettres mélangées. La plupart des CRDT et des algorithmes OT présentent ces anomalies ; Fugue et FugueMax les éliminent, FugueMax étant prouvé maximalement non-entrelaçant. Bonne nouvelle : Automerge utilisait RGA pour le texte avant de passer à FugueMax. Retenu : rien à faire, mais ajouter le scénario « deux pairs tapent au même endroit, dans les deux sens » à la simulation pour le vérifier sur notre version.

3. La traversée des NAT sans serveur. Combien de connexions directes réussiront ? La mesure de référence sur libp2p : un taux de succès conditionnel de 70 % ± 7,1 % pour le hole punching, identique en TCP et en QUIC, 97,6 % des succès dès la première tentative, 82,9 % pour la classe de NAT la plus courante. Donc 20 à 30 % des paires hors LAN passeront par un relais : il en faut un. Teamtype s'appuie sur les relais publics d'Iroh, croc sur un relais public de son auteur, Syncthing sur des relais communautaires. Retenu : un relais libp2p (`circuit v2`) que nous hébergeons, adresse configurable, auto-hébergeable en une commande, et le LAN sans relais comme chemin principal. Le relais ne voit que du chiffré.

4. Révocation concurrente et antidatage. Que faire si deux admins se révoquent mutuellement hors ligne, ou si un pair exclu antidate ses opérations ? Keyhive refuse le consensus : obtenir un accord sur les opérations acceptées jusqu'à un point puis rejeter le concurrent irait contre l'esprit local-first ; ils préfèrent garder le niveau de cohérence d'Automerge et traitent ces cas dans leur CRDT de groupe. Matrix résout le même problème avec un algorithme de résolution d'état déterministe et des niveaux de pouvoir ; Radicle avec un seuil de délégués signataires. Retenu pour la v1 : la liste de membres est un petit CRDT à ordre total déterministe (horloge de Lamport, puis clé publique) ; une révocation l'emporte sur une invitation concurrente ; les opérations causalement postérieures à une révocation sont refusées, les concurrentes acceptées et marquées dans l'Activité. Le vrai remède est Keyhive, quand il sera livré.

5. Le document survit-il à son créateur ? Matrix et Radicle répondent par plusieurs administrateurs. Retenu : rôle `admin` distinct ; l'hôte initial est admin ; l'interface pousse à nommer un second admin dès le premier partage ; un document sans admin joignable reste éditable mais ne peut plus changer ses membres.

6. Tampon de l'éditeur, CRDT et éditions externes. Ethersync/Teamtype a tranché par trois mécanismes : un module OT entre éditeur et démon, un module sandbox qui interdit toute E/S hors du dossier de base, et un watcher ; plus une règle de propriété : lors du scan du dossier, n'agir sur un changement que si le démon possède le fichier, l'éditeur étant propriétaire tant qu'il a le fichier ouvert. Chez nous l'éditeur est lié directement au document (`automerge-codemirror`), donc pas d'OT ; on reprend la règle de propriété pour `fsnotify` et le sandbox pour toutes les E/S.

7. Mémoire et performance d'Automerge. La réputation de lourdeur vient d'Automerge 1 : 154 Mo pour un document de 127 Ko de texte ; l'encodage en colonnes d'Automerge 2 ramène cela à environ 160 Ko, soit plus de 1000× de compression. Retenu : un rapport Typst de cent pages n'est pas un problème ; le test de charge de l'étape 5 fixe la limite réelle sur notre pile.

8. Débit du pont Wails. Personne n'a publié de mesure pour Automerge dans Wails ; Tauri a dû ajouter un canal binaire dédié pour ce genre de flux. Retenu : mesurer à l'étape 1 ; si les bindings saturent, WebSocket local avec jeton, déjà prévu.

9. Multicast bloqué. Fréquent en entreprise et sur le Wi-Fi invité. Syncthing combine découverte locale, serveurs de découverte globaux et relais ; nous n'aurons pas de découverte globale. Retenu : mDNS libp2p sur le LAN, saisie manuelle `ip:port` , et le code Wormhole comme chemin universel ; un message clair quand aucun pair n'est vu après dix secondes.

10. Fusion sémantique laide après hors-ligne. Aucune bibliothèque ne la résout ; Patchwork la rend visible (diff par auteur, suppressions au survol) et réversible (annulation sélective). Retenu : c'est la réponse, déjà planifiée en étape 4 avec

`@codemirror/merge`.

Reste sans réponse ailleurs : le commentaire ancré dans le PDF rendu et la minibar reliée à l'aperçu. Ce sont nos idées propres, à prototyper sans modèle.

Confort d'édition Typst

L'éditeur reste un éditeur de source : chaque bouton ou raccourci insère ou enveloppe du Typst visible, jamais une mise en forme cachée. C'est le modèle de typst.app, dont la barre permet de changer la police, souligner, mettre en italique ou en gras et changer le niveau de titre, et c'est aussi là que les utilisateurs demandent plus : sélectionner un passage et cliquer pour l'envelopper, comme TeXstudio le faisait, parce que taper `#quote[…]` deux cents fois est laborieux. Le plugin Obsidian Typst Book Preview montre la bonne règle : la barre enveloppe la sélection quand c'est possible et insère un extrait au curseur sinon.

Trois règles pour que les boutons soient fiables

1. *Connaître le mode* : en Typst, `*gras*` ne marche qu'en mode markup ; dans un bloc de code il faut `#strong[…]`, dans un bloc math `bold(…)`, dans du raw rien. Chaque action consulte l'arbre syntaxique au curseur (grammaire Lezer pour CodeMirror, ou les jetons sémantiques de tinymist) avant d'écrire.
2. *Basculer, pas empiler* : appliquer « gras » sur une sélection déjà en gras l'enlève ; la barre affiche l'état courant (gras actif, niveau de titre) comme un traitement de texte.
3. *Une seule source de vérité pour les raccourcis* : toutes les actions sont des commandes CodeMirror nommées, exposées dans la palette, la barre, le menu contextuel et les raccourcis configurables.

Catalogue des fonctions, et d'où elles viennent

Fonction	Typst produit	Fourni par	À écrire
Gras, italique, souligné, barré, surligné, indice, exposant, code inline	`*…*`, `_…_`, `#underline[…]`, `#strike[…]`, `#highlight[…]`, `#sub[…]`, `#super[…]`, `` `…` ``	—	Commandes d'enveloppe avec bascule et détection de mode ; raccourcis Ctrl+B/I/U, Ctrl+Shift+X, Ctrl+Shift+H, Ctrl+E

Fonction	Typst produit	Fourni par	À écrire
Niveaux de titre par sélection ou raccourci	`=` , `==` , …	—	Commande « définir le niveau n » (Ctrl+1…6, Ctrl+0 pour revenir au paragraphe) qui réécrit le préfixe de ligne
Sommaire interactif	—	tinymist : symboles de document, c'est-à-dire plan ou table des matières, avec repli des blocs et des titres ; en plus une vue plan du document exporté, différente du plan syntaxique	Panneau latéral : clic → saut, titre courant surligné pendant le défilement, glisser-déposer d'une section pour déplacer tout son bloc, renommage inline
Listes à puces, numérotées, de termes	`-` , `+` , `/ terme:`	—	Entrée continue la liste, Tab/Maj+Tab indente, Entrée sur un item vide sort de la liste ; bouton bascule puce/numéro
Citations, notes de bas de page, encadrés	`#quote[…]` , `#footnote[…]` , `#block[…]`	—	Enveloppes ; pour la note, curseur placé à l'intérieur
Liens, labels, références croisées	`#link("…")[…]` , `<label>` , `@label`	tinymist : complétion de `@` , renommage des symboles, aller à la définition	Ctrl+K colle l'URL du presse-papiers si c'en est une ; bouton « créer un label » sur un titre ou une figure

Fonction	Typst produit	Fourni par	À écrire
Symboles spéciaux et math	`$…$`, `sym.alpha`, `→`	tinymist : vue des symboles, recherche par mot-clé, description ou tracé à la main, groupés par catégorie, insertion au clic ; complétion des symboles en math	Panneau symboles branché sur la ressource tinymist, raccourci Ctrl+Shift+S, favoris, insertion sous forme Unicode ou `sym.` selon le mode
Typographie automatique (guillemets, tirets, points de suspension)	`--`, `---`, `...`, guillemets droits	Typst lui-même à la compilation	Rien à faire ; ne pas doubler. Ajouter seulement l'espace insécable `~` (Ctrl+Espace) et un indicateur discret des espaces insécables
Images et figures	`#figure(image("…"), caption: […])`	magasin de blobs	Glisser-déposer et coller depuis le presse-papiers : copie dans les ressources du projet, insertion de la figure, curseur dans la légende
Tableaux	`#table(columns: 3, […], …)`	—	Petite grille « lignes × colonnes » au clic ; action « aligner les cellules du tableau » pour garder la source lisible
Complétion, extraits, documentation au survol, diagnostics inline, couleurs	—	tinymist : complétion, code lens, pastilles de couleur avec sélecteur, survol	Client LSP (`codemirror-languageserver`) et rendu Adwaita des infobulles

Fonction	Typst produit	Fourni par	À écrire
Formatage du document	—	tinymist : reformatage via typstfmt ou typstyle	Bouton et option « formater à la sauvegarde »
Paires automatiques	`[]`, `()`, `$$`, `*`, `_`, `"`	CodeMirror `closeBrackets`	Configuration des paires Typst, avec exception quand le caractère suivant est une lettre (pour `*` et `_`)
Recherche et remplacement, multi-curseur, pliage	—	CodeMirror	Pliage par titre fondé sur le plan
Correcteur orthographique	—	`spellcheck` natif de la webview (WebKit, WebView2) sur le contenu éditable	Désactivé dans le code et le math via l'arbre syntaxique ; dictionnaires système, donc gratuit sur les trois plateformes
Compteur de mots, temps de lecture	—	compilé par Typst (`typst query`) ou approximation sur la source	Barre d'état ; comptage sur la sélection
Modèles de documents	—	tinymist : galerie de modèles et initialisation	Écran « nouveau document »
Réglages de page et police	`#set page(…)`, `#set text(font: …)`	tinymist : vue des polices	Dialogue qui lit et réécrit le bloc `#set` en tête de fichier
Mode concentration, machine à écrire	—	CodeMirror	Masquer la barre latérale, centrer la ligne courante, réduire la coloration

Fonction	Typst produit	Fourni par	À écrire
Palette de commandes	—	—	Ctrl/Cmd+K, liste toutes les commandes CodeMirror et application

Ce qu'il faut acheter ou vérifier avant l'étape 3

- Une grammaire Lezer Typst pour CodeMirror : il en existe dans la communauté, à évaluer sur la détection markup/code/math/raw ; sinon, jetons sémantiques tinymist pour la couleur et un détecteur de mode maison, plus simple qu'il n'y paraît.
- L'accès aux vues tinymist (symboles, polices, plan exporté) depuis un client autre que VS Code : elles passent par des commandes LSP du serveur ; à confirmer dans la doc de tinymist pour chaque vue.
- Le rendu des infobulles LSP en Adwaita : Markdown de tinymist à convertir avec `markdown-it`, déjà prévu pour Markdown.

Idées des concurrents à reprendre : Typst Desktop propose une barre de formatage et des réglages de page, un navigateur de ressources avec aperçus dans la barre d'outils, et une bibliothèque de ressources partagée entre projets. Typwriter et Typeset confirment que la synchro source↔aperçu dans les deux sens, l'autocomplétion et le survol sont désormais le minimum attendu.

Autocomplétion, y compris sur nos paquets et chartes

tinymist complète tout ce qu'il voit, sans distinction entre Universe et le code de l'utilisateur :

- Les `#let` du fichier et des fichiers importés, avec aide à la signature (paramètres nommés, valeurs par défaut, types déduits) ; les commentaires de documentation `///` au-dessus d'une fonction alimentent le survol et la liste.
- Les paquets locaux (`@local/charte-acme:1.0.0` dans le dossier de paquets de l'utilisateur) et les paquets vendus dans le projet : complétion dès `@`, puis des fonctions exportées et de leurs paramètres.
- Les valeurs : polices installées, couleurs nommées, labels pour `@ref`, chemins de fichiers du projet pour `#image`, `#include`, `#read`.

Ce que l'éditeur ajoute pour une charte graphique :

1. *Panneau « Charte »* : généré automatiquement à partir des exports du paquet de charte déclaré dans le projet (`charte: "@local/charte-acme:1.0.0"` dans la config) et de leurs doc-comments ; chaque composant (page de garde, encadré, tableau maison, en-

tête de section) est affiché avec un aperçu rendu via tinymist et s'insère au clic avec ses paramètres obligatoires pré-remplis. Une charte bien documentée se branche sans code.

2. *Extraits de projet* : fichier `.collab/snippets.typ` synchronisé, déclencheur + corps avec tabulations (`enc` → `#encadre(titre: "${1}")[${2}]`), fusionnés avec les extraits de tinymist dans la même liste de complétion.
3. *Modèle lié à la charte* : « nouveau document » propose les modèles du paquet de charte avec l'import et le `#show` déjà posés.
4. *Paquet local partagé* : la charte vendue dans le projet (section Ressources) voyage avec lui, donc tous les pairs ont la même version et la même complétion ; une mise à jour de la charte est un changement ordinaire dans l'Activité.
5. *Aide à l'écriture de la charte* : pour l'auteur du paquet, un gabarit `typst.toml` + `lib.typ` documenté, et le linter de tinymist pour vérifier que chaque fonction exportée a un doc-comment, condition pour apparaître correctement dans le panneau.

Orthographe, indentation, coloration

Fonction	Niveau 1 (gratuit, v1)	Niveau 2 (intégré, étape 4)	Niveau 3 (si demandé)
Orthographe et grammaire	Attribut `spellcheck` de la webview : dictionnaires système, français inclus, soulignement et suggestions natifs ; désactivé dans code, math et raw via l'arbre syntaxique	`nspell` + `dictionary-fr` / `-en` dans la webview : contrôle total, dictionnaire personnel par projet (termes techniques, noms de produits), mots ignorés partagés dans l'index Automerge	Vérificateur LSP à côté de tinymist : Harper (hors ligne, rapide, connaît la syntaxe Typst ; surtout anglais aujourd'hui) ou `ltex-ls-plus` (LanguageTool, grammaire française complète, lourd car Java)

Fonction	Niveau 1 (gratuit, v1)	Niveau 2 (intégré, étape 4)	Niveau 3 (si demandé)
Indentation	CodeMirror `indentOnInput` et `indentService` : Entrée après `[ ( {` indente, la fermeture désindente, listes continuées au niveau courant ; `indentUnit` configurable (2 espaces par défaut)	Dans les blocs raw, délégation au langage imbriqué (Go, JSON, YAML, SQL…) ; Tab et Maj+Tab sur une sélection multi-lignes	Formatage global via tinymist/typstyle : bouton, raccourci et option « à la sauvegarde » ; alignement automatique des tableaux `#table`
Coloration	Grammaire Lezer Typst dans CodeMirror : instantanée, hors ligne, distingue markup / code / math / raw ; parenthèses appariées, ligne courante, résultats de recherche	Jetons sémantiques tinymist par-dessus (fonction vs variable, paramètre nommé, label), occurrences du symbole sous le curseur, pastilles de couleur sur les littéraux	Coloration imbriquée des blocs de code dans leur propre langage (`@codemirror/language-data`, chargement à la demande), mêmes thèmes que le PDF (zebraw/codly) pour que l'éditeur ressemble à la sortie

Décision : niveau 1 partout dès l'étape 3, parce qu'il ne coûte presque rien ; niveau 2 à l'étape 4 ; niveau 3 à la demande des utilisateurs, la grammaire française en LSP étant la seule pièce coûteuse.

Orientation IT : documentation technique, rapports, runbooks

Le public visé écrit des specs, des rapports d'audit, des procédures, des README longs. Ce qu'il attend va au-delà du gras et des titres, et Typst Universe fournit déjà la plupart des blocs ; l'éditeur doit les rendre accessibles sans connaître les paquets.

Besoin	Paquet ou mécanisme Typst	Ce que l'éditeur ajoute
Blocs de code colorés avec numéros de ligne, lignes surlignées, commentaires en marge, en-tête avec nom de fichier	Coloration native de Typst dans les blocs raw ; codly, modèle de présentation de code avec indentation intelligente, numérotation, surlignage ; ou zebraw, plus léger et plus rapide, avec numéros de ligne, surlignage de lignes et commentaires explicatifs, en-têtes et pieds, découpe par plage de lignes	Bouton « bloc de code » avec sélecteur de langage (mémorise le dernier), coloration *imbriquée* dans l'éditeur (CodeMirror sait charger la grammaire du langage à l'intérieur du bloc raw, comme pour Markdown), Tab insère l'indentation du langage, collage depuis un IDE conserve tout. Le choix codly ou zebraw est un réglage de projet ; zebraw par défaut pour les gros documents, codly restant plus riche mais nettement plus lent avec de nombreux blocs
Inclure un fichier source du projet plutôt que coller	`#raw(read("src/main.go"), lang: "go", block: true)`	Glisser un fichier de code dans l'éditeur propose « inclure » (lien vivant, toujours à jour) ou « coller » (copie figée) ; blocs partagés par le magasin de blobs
Blocs terminal, logs, sorties de commande	raw avec `lang: "console"` ou `"log"`, style distinct	Préréglage « terminal » : fond sombre, prompt non sélectionnable, pas de numéros
Encadrés d'avertissement, note, astuce, danger	gentle-clues, admonitions prêtes à l'emploi, ou showybox	Bouton « encadré » avec les quatre variantes et icônes Lucide ; raccourci `>` + mot-clé en début de ligne

Besoin	Paquet ou mécanisme Typst	Ce que l'éditeur ajoute
Diagrammes d'architecture, flux, séquences	fletcher : nœuds et arêtes, CeTZ pour le dessin libre, conversion d'ASCII art en SVG, icônes de topologie réseau pour CeTZ/fletcher	Extraits de départ (3 nœuds reliés), aperçu du diagramme seul au survol via tinymist, et import d'un SVG exporté depuis draw.io ou Excalidraw comme figure
Tableaux techniques (paramètres, matrices de compatibilité, API)	`#table` ; styles booktabs via un paquet de la galerie	Grille au clic, alignement automatique de la source, coller un CSV ou un tableau Markdown convertit en `#table`
Touches clavier, chemins, variables, noms de fichiers	code inline `` `…` `` et une fonction `#kbd[…]` de modèle	Raccourci Ctrl+E ; détection d'un chemin ou d'une commande collée qui propose le code inline
Arborescences de fichiers	paquet d'arbre de fichiers de la galerie, ou liste imbriquée en monospace	Coller la sortie de `tree` la convertit
Références de sécurité, CVE, RFC, tickets	`#link` avec préfixes connus	Autoliens : `CVE-2026-1234`, `RFC 9114`, `#1234` deviennent des liens selon les préfixes configurés par projet
Numéros de version, dates, statut du document	`#set document(...)`, variables en tête de fichier	Dialogue des métadonnées ; bandeau « brouillon / relu / publié » tiré d'une variable, repris dans le PDF
Glossaire et acronymes	paquet d'abréviations de la galerie	Complétion des acronymes définis, bouton « ajouter au glossaire » sur la sélection

Besoin	Paquet ou mécanisme Typst	Ce que l'éditeur ajoute
Export pour un wiki ou un site	export HTML de Typst, paquet pour blocs de code sans en-tête optimisés pour l'HTML	Export HTML et Markdown en plus du PDF, pour coller dans Confluence, GitLab ou un site statique

Modèle de projet « documentation technique » : un modèle livré avec l'application qui préconfigure zebraw, gentle-clues, booktabs, les polices à chasse fixe, la numérotation des figures et la page de garde, pour que le premier document d'un ingénieur soit beau sans toucher à un `#import`. Les modèles passent par la galerie tinymist, déjà prévue.

Deux détails qui comptent pour ce public : la police de code doit avoir des ligatures désactivables (JetBrains Mono, Adwaita Mono) et le PDF doit conserver le texte des blocs de code copiable tel quel, sans numéros de ligne collés ; zebraw et codly le gèrent, à vérifier dans le test PDF de référence.

Effet sur le plan : l'étape 3 inclut la barre contextuelle, les titres, les listes, le plan, les blocs de code colorés et le niveau 1 d'orthographe, indentation et coloration ; symboles, tableaux, images, encadrés, extraits de projet et panneau Charte passent en étape 4 ; diagrammes, autoliens, glossaire, export HTML, correcteur intégré et réglages de page en étape 6.

Interface du projet : fichiers, état, personnes

Le produit n'est pas un éditeur de fichier mais un espace de travail partagé. Trois panneaux fixes portent cette idée : l'explorateur (quoi), l'état de synchronisation (où en est-on) et les personnes (avec qui). Ils suivent la même règle que l'éditeur : clair par défaut, détails techniques à la demande.

Explorateur de projet

Le projet est l'unité qu'on partage, donc l'interface doit le montrer en entier, pas seulement le fichier ouvert. Un panneau latéral « Fichiers » est présent dès la v1.

- *Arborescence complète* : tous les fichiers du dossier, pas seulement les `.typ` — images, polices, bibliographie `.bib`, données CSV/JSON, paquets vendus, PDF exportés. Les fichiers texte s'ouvrent dans l'éditeur (Typst, Markdown, texte, code, avec la coloration du langage), les images et PDF dans un visualiseur, les polices avec un spécimen, les CSV dans une grille en lecture seule.

- *État de synchronisation par fichier* : pastille « à jour », « en cours », « modifié par X il y a 2 min » (tiré de l'historique Automerge), « nouveau depuis ma dernière session ». Un fichier qu'un pair édite en ce moment affiche son avatar.
- *Opérations* : créer, renommer, déplacer, dupliquer, supprimer, révéler dans le gestionnaire de fichiers système, glisser-déposer depuis l'extérieur. Renommer et déplacer sont des opérations sur l'index Automerge du projet, donc synchronisées et fusionnées comme le reste ; un renommage concurrent du même fichier se résout par l'ordre déterministe et se signale dans l'Activité. Supprimer déplace à la corbeille du projet (récupérable), jamais de suppression directe.
- *Fichier principal* : marqué d'une épingle ; c'est celui que tinymist compile pour l'aperçu et l'export. Changement par clic droit, synchronisé lui aussi.
- *Filtrage* : règles d'exclusion partagées (`.gitignore` respecté, `.collab/` , caches, sorties PDF), recherche par nom, recherche plein texte dans le projet via l'index local.
- *Vignettes* : première page compilée pour les `.typ` , miniature pour les images, comme le fait Typst Desktop ; calculées à la demande et mises en cache.
- *Liens avec l'éditeur* : les chemins dans `#image("…")` , `#include` , `#read` et `#bibliography` sont cliquables, complétés par tinymist, et surlignés en erreur si la cible n'existe pas. Glisser un fichier de l'explorateur dans l'éditeur insère l'appel adéquat (`#image` , `#include` , `#raw(read(…))`).
- *Dossiers volumineux* : un dossier de plus de quelques milliers de fichiers ou de plusieurs centaines de Mo déclenche un avertissement et propose d'exclure des sous-dossiers ; l'application est un éditeur de documents, pas un outil de sauvegarde.

Lisibilité de l'état : synchro, pairs, branches

Un outil sans serveur n'a pas de « cloud » qui rassure ; l'interface doit remplacer cette confiance par de la clarté. Principe : à tout moment, en un coup d'œil, l'utilisateur sait *avec qui* il est connecté, *si* tout le monde a la même chose, et *sur quelle version* il travaille. Jamais de jargon (pas de « heads », « CRDT », « hole punching ») dans l'interface par défaut.

- *Indicateur global dans la barre d'en-tête* : une pastille et un mot. « Synchronisé » (vert : tous les pairs connectés ont le même état), « En cours » (bleu animé : des changements voyagent), « Hors ligne, N changements en attente » (gris : rien de perdu, ils partiront au retour), « Attention » (orange : un pair n'a pas reçu nos changements depuis plus de X minutes alors qu'il est connecté, ou un pair tente d'écrire sans droit). Cliquer ouvre le panneau d'état.
- *Panneau d'état* : la liste des pairs avec avatar, couleur, machine, type de lien (LAN direct, Internet direct, via relais, hors ligne depuis…), latence, et pour chacun « à jour » ou « en retard de N changements ». Un pair hors ligne reste listé en grisé avec l'heure de

sa dernière synchro. Boutons : inviter, copier le code, retirer un membre, passer en lecture seule.

- *Vue par fichier* : reprise de l'explorateur (pastille par fichier) et, dans l'éditeur, une bande fine en haut quand le fichier ouvert a reçu des changements distants non encore lus par l'utilisateur, avec « voir ce qui a changé » qui ouvre le diff coloré par auteur.
- *Branches, quand elles existeront (étape 4 ou v2)* : le nom de la branche courante toujours visible à côté du titre du document, « main » par défaut. Un sélecteur montre les branches, leur auteur, leur dernière activité, le nombre de changements d'écart avec `main`, et deux actions seulement : « comparer » (diff dans l'éditeur) et « fusionner dans main ». Pas de rebase, pas de conflit à résoudre à la main, la fusion est toujours possible ; l'utilisateur relit le résultat comme un diff ordinaire. Une branche privée non diffusée (le brouillon d'Upwelling) porte une icône d'œil barré pour qu'on sache qu'elle n'est pas partagée.
- *Historique lisible* : le panneau Activité (section Fusion) raconte en phrases : « Marie a modifié l'introduction il y a 10 min », « Jalon : version envoyée au client », « Fusion de la branche *refonte-plan* ». Un curseur temporel permet de voir le document à n'importe quel moment, en lecture seule, et d'en repartir (« créer une branche d'ici » ou « restaurer ce fichier »).
- *Erreurs expliquées* : chaque état d'alerte vient avec une phrase et une action. « Impossible de joindre Paul directement ; connexion par relais » plutôt qu'un code ; « Le paquet `@preview/zebraw` n'est pas disponible hors ligne — réessayer dès que le réseau revient » avec le bouton correspondant.
- *Première minute* : à l'ouverture d'un projet partagé, une courte bannière rappelle qui a accès, puis disparaît. À la première perte de réseau, une infobulle explique une fois que tout continue de fonctionner et se synchronisera au retour.
- *Mode détaillé* : un interrupteur « détails techniques » dans le panneau d'état affiche les identifiants, les vecteurs d'état, les adresses et les logs réseau pour le débogage et les rapports de bug, sans encombrer l'affichage par défaut.

Présence en direct et gestion des personnes

Voir les autres travailler est la raison d'être du produit ; la gestion des personnes doit donc être aussi soignée que l'éditeur, et rester simple : des gens, pas des comptes.

Ce qu'on voit pendant l'édition

- Curseur et sélection de chaque pair dans sa couleur, avec une étiquette nom qui apparaît au mouvement puis s'estompe (modèle Google Docs / Figma). Les sélections distantes sont translucides pour ne jamais masquer le texte. Tout cela vient des messages éphémères Automerge, déjà dans `automerge-codemirror`.

- Dans la gouttière, une fine barre colorée sur les lignes qu'un pair a modifiées dans les dernières secondes, qui s'efface progressivement : on voit *où* ça bouge sans lire.
- Pile d'avatars en haut du document : qui est dans *ce* fichier ; au survol, le nom et la machine ; en gris, ceux qui sont dans le projet mais sur un autre fichier, avec le nom du fichier. Cliquer sur un avatar fait défiler jusqu'à son curseur.
- Mode « suivre » : on s'accroche au curseur d'un pair et l'éditeur suit ses déplacements, utile pour la relecture à deux ou pour montrer quelque chose. Un clic ou une frappe en sort.
- Indicateur « en train d'écrire » dans la pile d'avatars et dans la liste des pairs, pour savoir si quelqu'un est actif ou simplement connecté.
- Dans l'aperçu PDF, les curseurs des autres sont projetés à la page correspondante (via les spans de tinymist) sous forme de petits repères colorés dans la marge.
- Dans l'explorateur, l'avatar du pair à côté du fichier qu'il édite.

Les personnes

- Un profil local : prénom ou pseudo, couleur choisie dans la palette Okabe-Ito, avatar optionnel (image locale, jamais téléchargée ailleurs ; transmise aux pairs comme blob). Créé au premier lancement en trois champs, modifiable n'importe quand.
- Une personne peut avoir plusieurs machines ; les pairs les regroupent sous le même nom et la même couleur, avec une puce « portable » ou « bureau ». Techniquement, chaque machine a sa clé ; le regroupement est déclaré par l'utilisateur et signé par ses propres clés, modèle des groupes d'appareils de Keyhive.
- Pas de serveur d'identité : le nom est ce que la personne déclare, l'empreinte de clé est ce qui garantit que c'est bien elle. L'empreinte s'affiche sous forme courte et prononçable (quatre mots) pour pouvoir la vérifier à voix haute une fois ; après vérification, une coche verte reste.
- Deux noms identiques dans un projet sont distingués automatiquement par un suffixe (« Marie · portable », « Marie B. »), jamais confondus dans l'historique.

Entrer et sortir

- Invitation par code ou par découverte LAN ; l'hôte voit « Paul (empreinte : brise-lampe-cerise-nord) demande à rejoindre » avec Accepter en écriture / Accepter en lecture / Refuser. Les demandes refusées trois fois d'une même clé sont ignorées silencieusement.
- La liste des membres montre pour chacun : rôle, dernière présence, nombre de changements, machines. Changer un rôle ou retirer quelqu'un est une action à un clic, avec une phrase de confirmation qui dit clairement ce qui se passe (« Paul gardera sa copie actuelle mais ne recevra plus rien »).

- Un pair retiré voit chez lui une bannière neutre « Vous ne faites plus partie de ce projet » et garde l'accès en lecture à sa copie locale.
- Mentions `@nom` dans les commentaires et l'Activité, avec complétion sur les membres ; une mention produit une notification locale discrète chez la personne quand elle est connectée.

Discrétion

- Un bouton « me rendre invisible » coupe l'envoi du curseur et de la sélection tout en continuant à synchroniser le texte ; les autres voient l'avatar avec une icône d'œil barré. Réponse au stress d'être observé relevé par Upwelling, moins radicale qu'une branche privée.
- La présence n'est jamais enregistrée : qui regardait quoi et quand disparaît avec la session ; seul ce qui a été écrit reste dans l'historique.

Présentations

Typst fait de bonnes présentations : les paquets Touying et Polylux produisent des diapositives paginées avec thèmes, animations par étapes (`#pause`), notes d'orateur et numérotation, le tout dans la même source que le reste du projet. Un document de présentation est donc un fichier `.typ` ordinaire, synchronisé et édité à plusieurs comme les autres ; ce que l'application ajoute, c'est la façon de le *jouer*.

Création

- Modèle « Présentation » dans « nouveau document », avec Touying préconfiguré et la charte du projet appliquée (couleurs, polices, logo) ; Polylux en option pour qui le connaît.
- Dans l'éditeur, le plan latéral affiche une diapositive par entrée au lieu des titres, avec une vignette ; glisser-déposer réordonne les diapositives (déplacement du bloc source).
- Barre contextuelle adaptée : nouvelle diapositive, étape (`#pause`), notes d'orateur, colonne, image plein écran, bloc de code (zebraw, en police plus grande).
- Aperçu en mode « grille » : toutes les diapositives en miniatures, clic pour sauter dans la source.

Mode présentation

- Un bouton et un raccourci (F5 / Cmd+Maj+Entrée) basculent en plein écran, interface masquée, diapositive rendue par le compilateur (SVG vectoriel, donc net sur n'importe quel écran).
- Navigation : → ← Espace Page↓ Page↑, clic ou molette, Début/Fin, saisie d'un numéro puis Entrée ; les étapes `#pause` sont des pas comme les autres.

- Vue présentateur sur un second écran : diapositive courante, suivante, notes d'orateur, chronomètre et heure, barre de progression ; détection automatique de l'écran externe, choix manuel sinon.
- Pointeur laser (touche L), surlignage temporaire au trait (touche D) effacé au changement de diapositive, écran noir (B) et blanc (W), vue d'ensemble en grille (G) pour sauter.
- Télécommande : le téléphone ouvre une page servie sur le LAN (code QR affiché dans la vue présentateur) avec précédent/suivant, notes et chronomètre ; même mécanisme que l'aperçu partagé, lecture seule, expire à la sortie du mode.
- Transitions sobres (fondu, aucune) ; pas d'effets.

Présentation à plusieurs

- Mode « suivre le présentateur » : les pairs connectés voient la diapositive courante changer chez eux ; un coprésentateur peut prendre la main, une seule personne pilote à la fois. C'est la présence Automerge appliquée au numéro de diapositive.
- Les invités sans l'application suivent dans leur navigateur via le lien d'aperçu partagé.
- Pendant la présentation, l'éditeur reste utilisable sur une autre machine : corriger une faute en direct met à jour la diapositive chez le présentateur à la prochaine recompilation, sans quitter le plein écran.

Export : PDF une diapositive par page (avec ou sans étapes dépliées), PDF « notes » deux par page avec les notes d'orateur, images PNG par diapositive pour les coller ailleurs.

Principes UX/UI

Ce qui rend l'ensemble convivial n'est pas la somme des fonctions mais quelques règles tenues partout.

1. *Une seule fenêtre, trois zones* : explorateur à gauche, éditeur au centre, aperçu à droite ; chaque zone se replie d'un geste et se souvient de son état par projet. Rien ne s'ouvre dans une fenêtre flottante sauf les dialogues système.
2. *Le texte d'abord* : chrome minimal, barre d'en-tête Adwaita avec le titre, l'état de synchro et les avatars, c'est tout ; la barre de formatage apparaît sur la sélection et disparaît ; les panneaux secondaires (symboles, charte, activité, commentaires) partagent un même tiroir à droite à onglets.
3. *Tout a trois accès* : clic, raccourci, palette (Ctrl/Cmd+K). La palette montre le raccourci à côté de chaque commande, c'est ainsi qu'on les apprend.
4. *Zéro jargon par défaut* : « personne », « projet », « jalon », « synchronisé » ; les termes techniques vivent derrière l'interrupteur « détails ».

5. *Chaque message dit quoi faire* : une phrase, une action, jamais un code d'erreur seul ; les états d'attente ont une durée ou une progression visible.
6. *Réversible avant d'être confirmé* : supprimer, retirer un membre, fusionner une branche s'annulent pendant quelques secondes (toast « Annuler ») plutôt que d'exiger une confirmation préalable ; la corbeille et l'historique rattrapent le reste.
7. *Même langage visuel que le résultat* : la police de code de l'éditeur, les couleurs des blocs et des encadrés reprennent celles du PDF produit par la charte, pour que la source ressemble au livrable.
8. *Première minute soignée* : nom, couleur, un projet d'exemple ouvert, l'aperçu qui se met à jour dès la première frappe ; aucune installation, aucun compte, aucune question réseau avant le premier partage.
9. *Clavier complet, lecteur d'écran, contraste* : chaque action est atteignable sans souris ; focus visible ; cibles de 44 px ; palette de couleurs des personnes lisible par les daltoniens ; respect de `prefers-reduced-motion` et du thème système.
10. *Mesurer l'usage sans télémétrie* : tests utilisateurs en personne sur cinq tâches types (créer, partager, relire, présenter, exporter) à chaque étape du plan ; le temps pour réussir chacune est l'indicateur de convivialité.

Références de design assumées : libadwaita pour la structure et les composants, typst.app pour la relation source↔aperçu, Figma pour la présence, Google Docs pour les commentaires, Keynote pour le mode présentation.

Carte des fonctions et manques

Vue d'ensemble de tout ce que le document décrit, regroupé par domaine, avec l'étape du plan où chaque fonction arrive. Les lignes marquées **manque** n'étaient traitées nulle part ailleurs et sont ajoutées ici.

Domaine	Fonctions	Étape
Édition	CodeMirror thématisé, barre contextuelle (gras, italique, souligné, barré, surligné, indice, exposant, code), titres par raccourci, listes intelligentes, paires automatiques, recherche/remplacement, multi-curseur, pliage, palette de commandes	3

Domaine	Fonctions	Étape
Édition	Symboles (vue tinymist), tableaux au clic et depuis CSV/Markdown, images par glisser-coller, encadrés, extraits de projet, panneau Charte, mode concentration	4
Édition	Correcteur intégré avec dictionnaire de projet, grammaire LSP optionnelle, coloration imbriquée des blocs de code, autoliens, glossaire, réglages de page et police	6
Typst	tinymist résident, aperçu incrémental, saut source↔aperçu dans les deux sens, complétion, survol, diagnostics, formatage, export PDF déterministe, paquets vendus hors ligne, polices embarquées	3
Typst	Blocs de code zebraw/codly, inclusion vivante de fichiers source, blocs terminal, diagrammes fletcher/CeTZ, modèle « documentation technique », galerie de modèles	3–4
Typst	Export HTML/Markdown, archive de projet autonome, **manque : import** Markdown, DOCX et LaTeX vers Typst via pandoc, et **collage intelligent** (HTML copié depuis le web ou Word converti en Typst au collage)	6
Projet	Explorateur complet avec vignettes, états par fichier, corbeille, fichier principal épinglé, recherche plein texte, magasin de blobs par hachage, règles d'exclusion	2–3
Collaboration	Automerge, curseurs et sélections, pile d'avatars, mode suivre, invisible, hors-ligne puis fusion, lecture seule, commentaires ancrés, mentions, Activité par auteur et session, diff depuis la dernière synchro, minibar, undo local, jalons go-git	1–4

Domaine	Fonctions	Étape
Collaboration	Branches simples (comparer, fusionner), brouillon privé, curseur temporel, commentaires sur le PDF, **manque : fil de discussion léger** par projet (déjà possible dans le même document Automerge, utile pour « tu relis la partie 3 ? » sans sortir de l'app)	4–v2
Réseau et confiance	Identité Ed25519, découverte LAN, code Wormhole, libp2p + relais hébergé, liste de membres signée, rôles écriture/lecture/admin, révocation, empreintes prononçables	2
État et confort	Indicateur global, panneau des pairs, bandes « changements non lus », erreurs expliquées, mode détails techniques, i18n fr/en, accessibilité, raccourcis configurables	2–6
Robustesse	Simulation à graine, rejeu d'historique, fuzz, tests de bout en bout, journal réseau circulaire, **manque : récupération après plantage** (réouverture des fenêtres et documents, rien à perdre puisque chaque frappe est persistée, mais il faut le montrer) et **sauvegardes locales** horodatées du fichier Automerge, vérification d'intégrité à l'ouverture et réparation depuis la dernière bonne copie ou depuis un pair	5
Distribution	Signature et notarisation, mises à jour signées, logs exportables, promesse de confidentialité, **manque : rapport de bug en un clic** (logs + graine + version, préparé localement, envoyé seulement si l'utilisateur confirme, jamais de contenu de document)	6

Domaine	Fonctions	Étape
Aide	**manque : référence Typst hors ligne** consultable dans l'app (la doc officielle est libre ; tinymist la sert déjà pour le survol), **visite guidée** du premier projet, et un **document d'exemple** qui montre code, encadrés, diagrammes et collaboration	6
Visionneuse	**manque : visionneuse PDF digne de ce nom** pour le résultat exporté (zoom, recherche dans le PDF, vignettes de pages, impression) ; l'aperçu tinymist est fait pour éditer, pas pour relire un livrable. PDF.js dans la webview couvre tout cela	4
Automatisation	**manque : compagnon en ligne de commande** : `app export <projet> --pdf`, `app join <code> --headless` pour qu'une machine de CI ou un serveur de documentation rejoigne un projet, compile et publie à chaque jalon ; même binaire Go, sans fenêtre. C'est le pont naturel vers le monde IT visé	v2
Automatisation	**manque : aperçu partagé pour un invité sans l'application** : tinymist peut servir la page d'aperçu sur le LAN ; un lien temporaire en lecture seule permet à quelqu'un de suivre une relecture dans son navigateur. Pas de serveur public : LAN seulement ou via le relais, lecture seule, expiration à la fermeture	v2
Présentations	Modèle Touying lié à la charte, plan par diapositive avec vignettes, mode plein écran avec navigation clavier et souris, vue présentateur sur second écran, pointeur et annotations, télécommande téléphone sur le LAN, suivi du présentateur par les pairs, export PDF et PNG	4 (mode plein écran et navigation en 3)

Trois cohérences à garder en vue

1. *Une seule source de vérité par chose* : le texte dans Automerge, les blobs dans le magasin par hachage, les membres dans la liste signée, les versions dans go-git comme projection. Chaque panneau lit l'une d'elles, aucun n'en invente une.
2. *Un seul vocabulaire* : « pair » dans le code, « personne » dans l'interface ; « projet » et non « dossier » ou « dépôt » ; « jalon » et non « commit » ; « branche » seulement quand les branches existent.
3. *Tout ce qui se voit se désactive* : présence, découverte LAN, relais, vignettes, correcteur, résumés automatiques ; le réglage est à côté de la fonction, pas dans un écran lointain.

Hors périmètre, décidé : application mobile, édition WYSIWYG du PDF, synchronisation via un compte en ligne, chiffrement au repos (v2 si Keyhive), édition native de RTF/DOCX, permissions par dossier.

Plan de prototypage et questions ouvertes

Six étapes, chacune livrant quelque chose qui tourne ; l'ordre met les risques techniques en premier et l'habillage en dernier.

1. **Noyau** : Wails v3 + CodeMirror + Automerge dans la webview, interface `Transport` en Go avec une implémentation TCP manuelle entre deux machines, curseurs visibles. Sortie : deux personnes tapent dans le même fichier.
2. **Réseau** : `go-libp2p` derrière `Transport`, découverte multicast, code Wormhole, identité Ed25519, liste de membres signée, persistance Automerge sur disque et projection `.typ`. Sortie : hors-ligne puis retour, deux réseaux différents.
3. **Typst** : tinymist en sidecar avec fichiers mémoire, iframe d'aperçu, saut source↔aperçu, export PDF par binaire embarqué, paquets vendus dans le projet. Sortie : le premier document utile de bout en bout.
4. **Collaboration** : commentaires ancrés, panneau Activité (par auteur, par session, jalons), diff « depuis ma dernière synchro », minibar, undo par auteur, lecture seule, blobs par hachage.
5. **Qualité** : simulation à graine en CI, rejeu d'historique, fuzz des décodeurs, tests de bout en bout, promesse de confidentialité rédigée.
6. **Produit** : style Adwaita, Markdown, go-git et remotes, i18n, signature et notarisation sur les trois plateformes, mises à jour signées.

Premier jalon mesurable : à la fin de l'étape 3, écrire ce document de conception à deux dans l'application.

Questions encore ouvertes

- [ ] Licence de l'application (libre ou non, et laquelle) : décide la façon de publier et le ton de la promesse de confidentialité.
- [ ] Nom du projet et de son format de dossier (`.collab/` ? autre).
- [ ] Rôle `admin` distinct de l'hôte initial, pour qu'un document survive au départ de son créateur.
- [ ] Traitement des updates hors-ligne d'un auteur révoqué entre-temps (accepter si antérieures à la révocation ?).
- [ ] Pont Wails : bindings suffisants pour le débit d'Automerge, ou WebSocket local dès le départ ? À mesurer à l'étape 1.
- [ ] Résumés automatiques de l'historique : modèle local acceptable en taille, ou fonction abandonnée ?
- [ ] Tiptap en vue riche Markdown : v2 ou jamais ?
- [ ] Taille maximale d'un blob et d'un projet.

Sources principales : Teamtype · Keyhive · Patchwork · Upwelling · Automerge cursors · typvend · Quarto typst-gather · typst-pack · bones-sim · Hatchet embedded · Obsidian privacy